

Ingeniería de Software 1

Resumen Completo con Diagramas y Ejemplos Visuales

UBA — Ingeniería Informática | Versión 4 — 2025

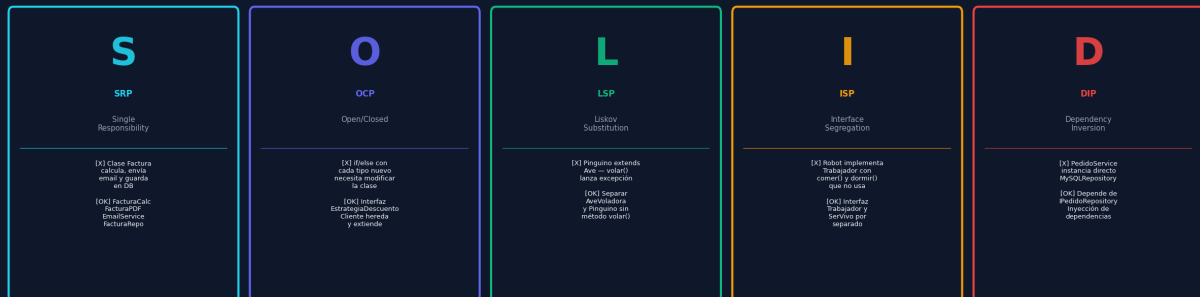
Este resumen incluye todos los temas del primer parcial ampliados con **diagramas visuales**, **gráficos de métricas** y **ejemplos de código** para los temas que los requieren en el examen.

1. Principios SOLID

Los principios SOLID son guías de diseño orientado a objetos que buscan **alta cohesión y bajo acoplamiento**. Un diseño que los respeta es extensible, mantenible y testeable.

Resumen Visual — los 5 principios

Principios SOLID — Referencia Rápida



S — Single Responsibility Principle (SRP)

Una clase tiene **una sola razón para cambiar**. Si una clase maneja múltiples responsabilidades, un cambio en una puede afectar las otras.

- **Pregunta clave:** ¿cuántas razones tiene esta clase para cambiar? Si más de 1 → viola SRP.
- **Señal de alarma:** clase con métodos de cálculo, persistencia, notificación y generación de reportes.

Ejemplo — Clase Factura antes y después

```
# [X] MAL: Factura hace TODO class Factura: def calcular_total(self): ... def generar_pdf(self): ... def enviar_email(self): ... def guardar_en_bd(self): ...

# [OK] BIEN: cada clase una responsabilidad class Factura: # solo lógica de negocio class FacturaPDFGenerator: # solo generación PDF class EmailService: # solo envío de mails class FacturaRepository: # solo persistencia
```

O — Open/Closed Principle (OCP)

Las clases deben estar **abiertas a la extensión, cerradas a la modificación**. Se implementa con polimorfismo o el patrón Strategy.

- **Señal de alarma:** un if/else que crece cada vez que se agrega un nuevo tipo.

```
# [X] MAL: agregar "empleado" requiere modificar esta clase class CalculadorDescuento: def calcular(self, tipo, monto): if tipo == "VIP": return monto * 0.80 elif tipo == "estudiante": return monto * 0.90 # agregar aquí → violar OCP # [OK] BIEN: nueva clase, no modifico nada existente from abc import ABC, abstractmethod class EstrategiaDescuento(ABC): @abstractmethod def calcular(self, monto): pass class ClienteVIP(EstrategiaDescuento): def calcular(self, monto): return monto * 0.80 class ClienteEmpleado(EstrategiaDescuento): # nueva, sin tocar nada def calcular(self, monto): return monto * 0.85
```

L — Liskov Substitution Principle (LSP)

Las subclases deben poder **reemplazar a su clase padre sin romper el comportamiento**. No basta con herencia conceptual; se necesita sustituibilidad real.

- [X] **Mal ejemplo:** Pinguino extiende Ave — volar() lanza excepción → LSP violado.
- [OK] **Solución:** Separar jerarquía: Ave (base), AveVoladora extiende Ave (con volar()), Pinguino extiende Ave (sin volar()).
- Alternativa: extraer interfaz *Volador* solo para las aves que vuelan.

I — Interface Segregation Principle (ISP)

No forzar a una clase a implementar métodos que no usa. Mejor tener **muchas interfaces específicas** que una grande.

```
# [X] MAL: Robot debe implementar comer() y dormir()
class Trabajador:
    def trabajar(self): ...
    def comer(self): ...
    def dormir(self): ...
# [OK] BIEN: interfaces segregadas
class Trabajador:
    def trabajar(self): ...
class Servivo:
    def comer(self): ...
    def dormir(self): ...
class Robot(Trabajador): ... # solo trabaja
class Humano(Trabajador, Servivo): ... # trabaja y es ser vivo
```

D — Dependency Inversion Principle (DIP)

Las clases deben depender de **abstracciones, no de clases concretas**. Se combina con inyección de dependencias.

```
# [X] MAL: acoplado a MySQL
class PedidoService:
    def __init__(self):
        self.repo = MySQLPedidoRepository()
# dependencia concreta
# [OK] BIEN: depende de la abstracción
class IPedidoRepository(ABC):
    @abstractmethod
    def guardar(self, pedido):
        pass
class PedidoService:
    def __init__(self, repo: IPedidoRepository):
        # inyección
        self.repo = repo
# Producción:
service = PedidoService(MySQLPedidoRepository())
# Tests:
service = PedidoService(MockPedidoRepository())
```

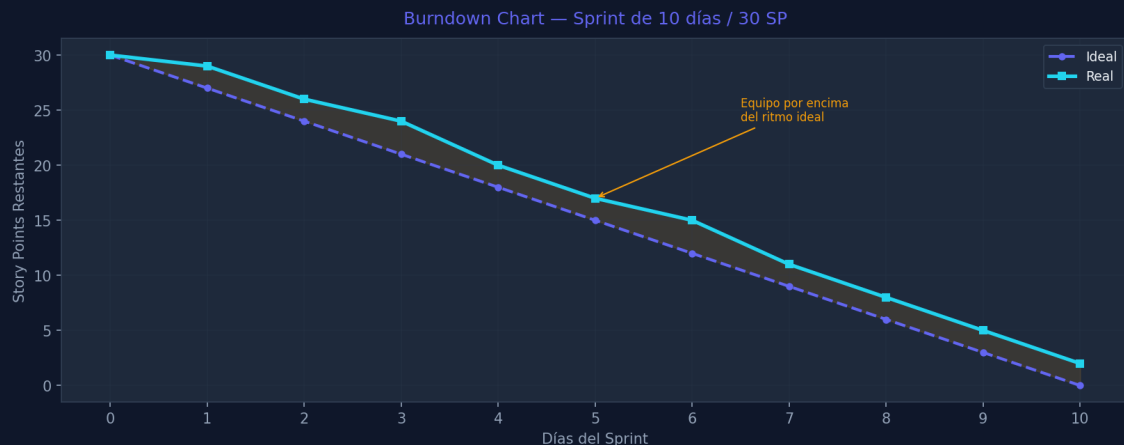
■ *DIP + Inyección de dependencias van casi siempre de la mano. Si ves `new ConcreteClass()` dentro de otra clase, hay violación de DIP.*

2. Métricas de Scrum — Diagramas

Scrum usa métricas visuales para trackear el progreso del sprint y la capacidad sostenida del equipo. Las dos principales son el **Burndown Chart** y el **Velocity Chart**.

Burndown Chart — Progreso del Sprint

Muestra los **story points restantes** vs **días del sprint**. La línea ideal decrece linealmente desde el total hasta cero. Si la línea real está por encima, el equipo está atrasado.

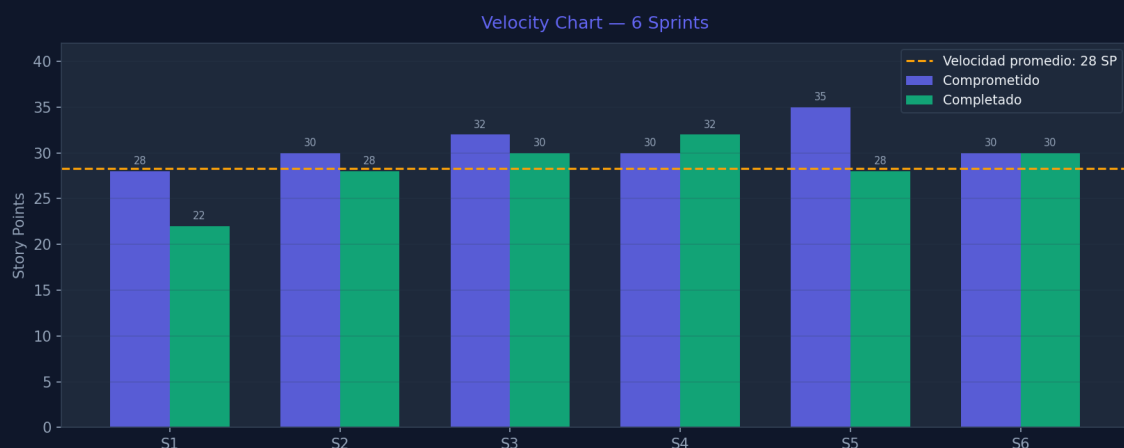


- **Eje X:** días del sprint (0 al 10 en un sprint de 2 semanas).
- **Eje Y:** story points pendientes de completar.
- **Línea ideal (punteada):** velocidad esperada uniforme.
- **Línea real:** progreso efectivo — puede ser irregular.
- **Área entre curvas:** deuda o adelanto respecto al plan.

■ Si la línea real sube (story points aumentan), se agregaron tareas al sprint. Si está muy por encima al final, es señal de sobreestimación o bloqueos.

Velocity Chart — Capacidad del Equipo

Muestra el trabajo completado por sprint vs lo comprometido, durante múltiples sprints. Permite estimar la **capacidad sostenida (velocidad promedio)** del equipo.



- **Barras azules:** story points comprometidos al inicio del sprint.
- **Barras verdes:** story points efectivamente completados.
- **Línea amarilla:** velocidad promedio — usada para planificar futuros sprints.

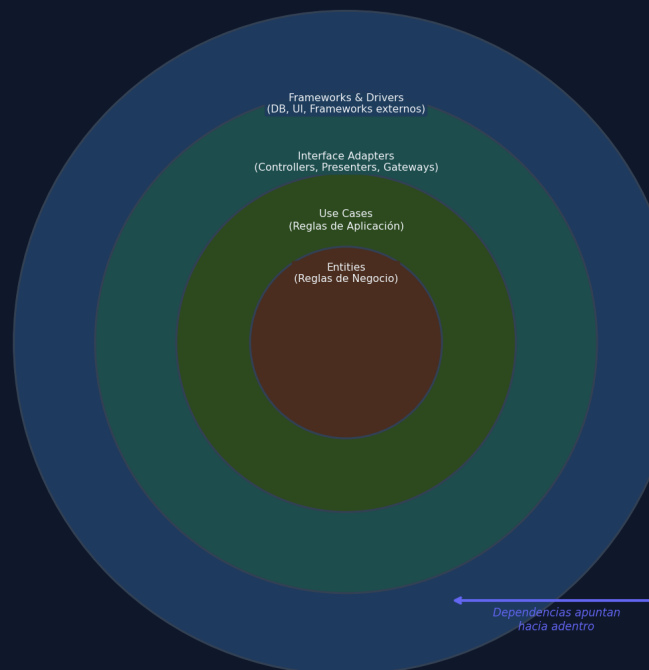
■ La diferencia entre comprometido y completado en S1 indica que el equipo sobreestimó su capacidad inicial. Con el tiempo se estabiliza.

3. Patrones de Arquitectura

Clean / Onion Architecture

La regla principal es que las **dependencias** siempre **apuntan hacia el centro**. Las capas internas no conocen las externas.

Clean / Onion Architecture



Capa	Contenido	Depende de	Ejemplo
Entities	Reglas de negocio empresa	NADA (núcleo)	Clase Pedido + validaciones
Use Cases	Reglas de la aplicación	Solo Entities	CrearPedidoUseCase
Adapters	Controllers, Gateways	Use Cases + Entities	PedidoController, RepoImpl
Frameworks	DB, UI, ORM, externos	Todo lo anterior	Spring, SQLAlchemy, React

- **Error más común:** poner lógica de negocio en el Controller o en el Repository.
- El Controller *solo* traduce HTTP a llamadas de Use Case. La lógica va en Use Cases o Entities.

4. Modelo de Vistas 4+1 (Kruchten)

Arquitectura descrita desde **5 perspectivas** distintas según la audiencia. La Vista de Escenarios (+1) coordina y valida las 4 vistas principales.



¿Cuándo usar cada diagrama UML por vista?

Vista	Diagrama UML	Audiencia	Responde...
Lógica	Clases, Objetos	Desarrolladores	¿Qué entidades hay?
Desarrollo	Componentes, Paquetes	Dev team	¿Cómo se organiza el código?
Procesos	Actividad, Secuencia	Integradores	¿Cómo fluye en runtime?
Física	Despliegue	Ops / Infra	¿Dónde corre el sistema?
Escenarios (+1)	Casos de uso	Todos	¿Qué hace el sistema?

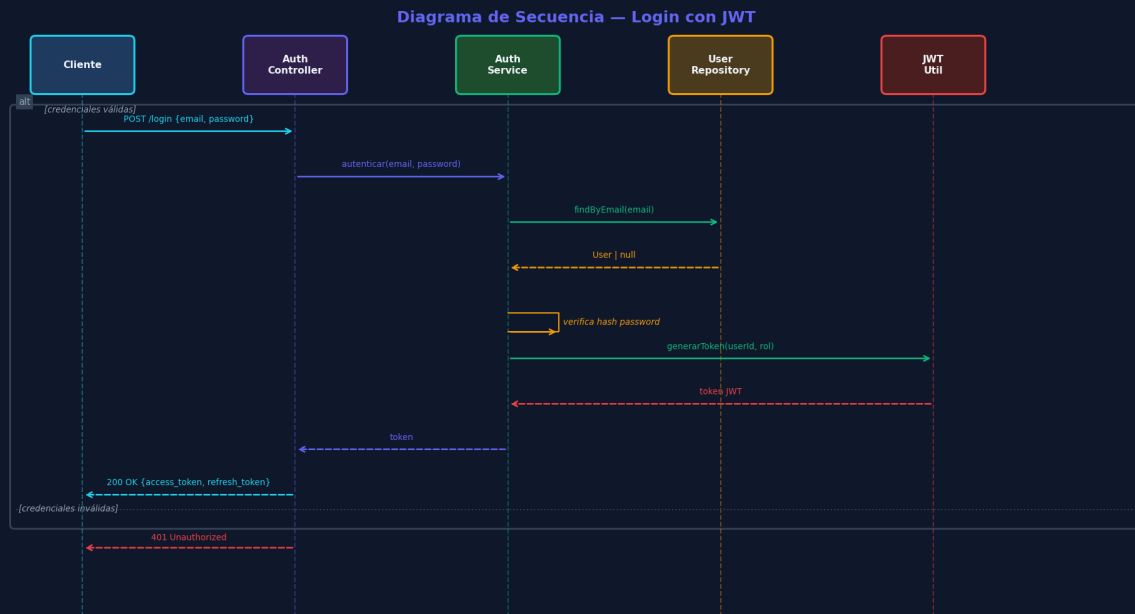
JSON Web Token es un estándar para transmitir información de forma segura entre partes como un JSON firmado digitalmente. Permite que el servidor sea **stateless**: no necesita consultar la DB para validar el token.

- **Si solo está firmado (no encriptado):** el payload es legible (base64) pero inalterable (firma detecta cambios).
- **Stateless:** el servidor valida el token con su clave privada, sin consultar DB.
- **Expiración:** el campo *exp* en el payload define cuándo vence.

6. Diagramas UML

Diagrama de Secuencia — Login con JWT

Muestra el flujo de mensajes entre objetos/capas para el caso de uso de login. Las flechas sólidas son llamadas, las punteadas son retornos. El fragmento **alt** modela los casos alternativos (error).

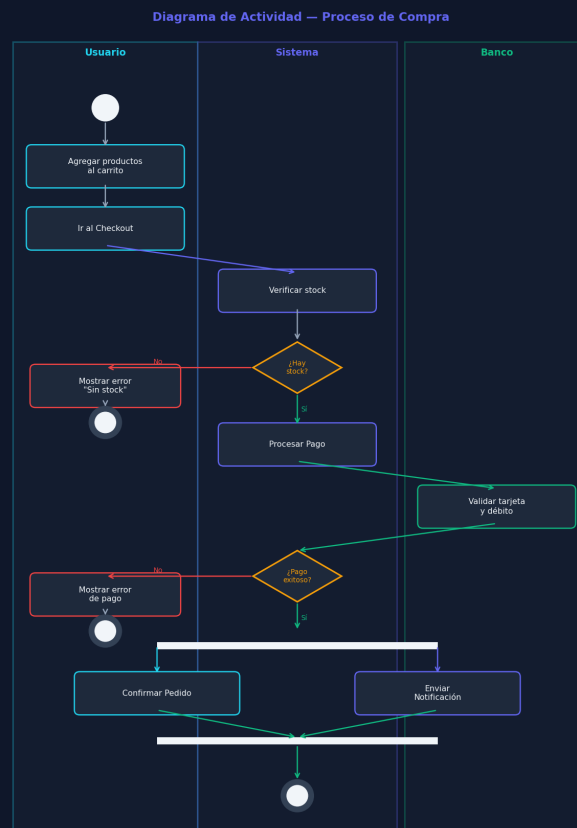


Reglas del Diagrama de Secuencia

- **Lifelines:** líneas verticales punteadas, una por participante.
 - **Mensajes síncronos:** flecha sólida con punta rellena →
 - **Retornos:** flecha punteada - - - →
 - **Fragmento alt:** para modelar condiciones alternativas (if/else del flujo).
 - **Fragmento loop:** para iteraciones.
 - **Fragmento opt:** para pasos opcionales.
- En el examen, si piden modelar el caso de error, usá el fragmento alt con [condición] en cada rama.

Diagrama de Actividad — Proceso de Compra

Modela el flujo de trabajo con **swim lanes** para separar responsabilidades por actor. Usa Fork/Join para actividades en paralelo.



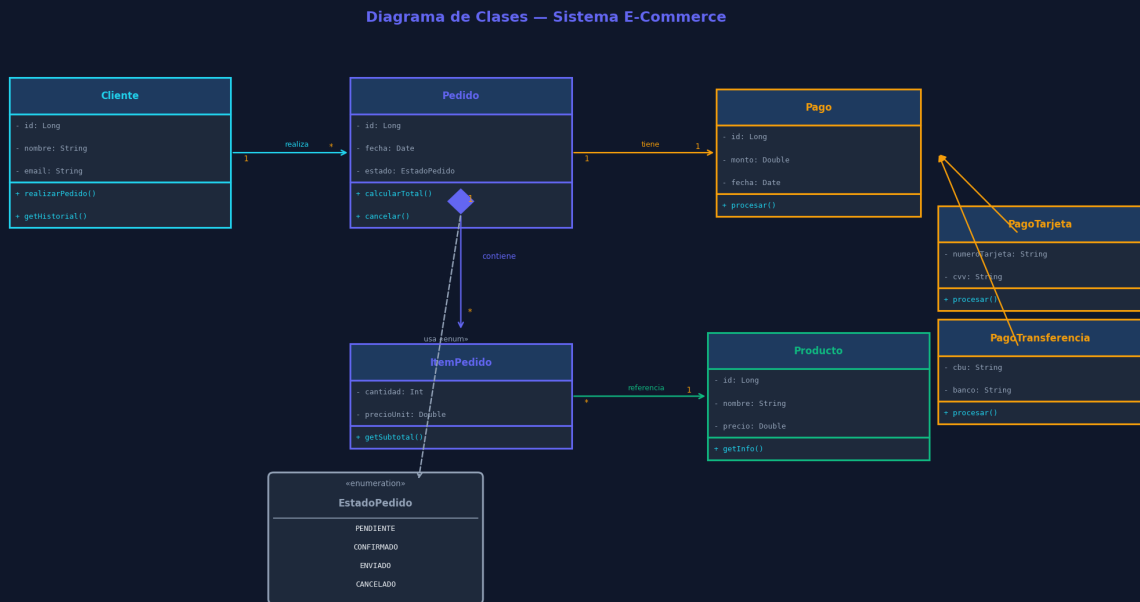
Elementos del Diagrama de Actividad

Elemento	Símbolo	Uso
Inicio	Círculo negro relleno ●	Punto de entrada al flujo
Fin	Círculo negro con borde ■	Fin del proceso
Acción	Rectángulo redondeado	Actividad concreta: "Verificar stock"
Decisión	Rombo ■	Bifurcación: [¿hay stock?] [¿pago exitoso?]
Fork	Barra gruesa horizontal ■■■	Inicio de actividades en paralelo
Join	Barra gruesa horizontal ■■■	Sincronización de paralelas
Swim Lane	Columna por actor	Separar: Usuario / Sistema / Banco

■ Si el enunciado dice "en paralelo" o "a la vez" → Fork/Join. Si dice "si/sino" o condición → Decisión (rombo).

Diagrama de Clases — Sistema E-Commerce

Modela la **estructura estática**: clases, atributos, métodos y relaciones. Es el diagrama más frecuente en exámenes de diseño.



Tipos de Relaciones — Resumen

Relación	Notación	Significado	Ejemplo
Asociación	————→	Un objeto conoce a otro	Cliente conoce Pedido
Agregación	<>————→	Todo-parte, parte puede existir sola	Universidad tiene Profesores
Composición	◆————→	Todo-parte, parte NO existe sin el todo	Pedido tiene ItemPedido
Herencia	< ——	Subclase extiende superclase	PagoTarjeta extends Pago
Realización	< ...	Clase implementa interfaz	MySQLRepo implements IRepo
Dependencia	...→	Uso temporal (parámetro o local)	Service usa DTO

■ Recordá siempre poner multiplicidades en todas las relaciones (1, *, 0..1, 1..*). Es un error muy frecuente en el parcial.

7. RESTful API — Referencia Rápida

HTTP Status Codes

HTTP Status Codes — REST Quick Reference

2xx — Éxito	3xx — Redirección	4xx — Error Cliente	5xx — Error Servidor
200 OK 201 Created 204 No Content	301 Moved Permanently 302 Found 304 Not Modified	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 409 Conflict	500 Internal Server Error 502 Bad Gateway 504 Gateway Timeout

Verbos HTTP

Verbo	Acción	Idempotente	Body	Ejemplo
GET	Obtener recurso	Sí	No	GET /users/123
POST	Crear recurso	No	Sí	POST /users
PUT	Reemplazar recurso completo	Sí	Sí	PUT /users/123
PATCH	Modificar parcialmente	No	Sí	PATCH /users/123
DELETE	Eliminar recurso	Sí	No	DELETE /users/123

Autenticación: Basic vs API Key vs Bearer

```
# Basic Auth – base64(user:pass) – NO encriptado Authorization: Basic dG9tYXM6cGFzc3dvcmQ= # API Key X-API-Key: abc123secretkey # Bearer (JWT o Opaque Token) Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Ejemplo completo: Login REST

```
# REQUEST POST /api/v1/users/token Authorization: Basic dG9tYXM6cGFzc3dvcmQ= # RESPONSE EXITOSA (200 OK) { "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "refresh_token": "m9obiRTw2CNklrA..." } # RESPONSE FALLIDA (401 Unauthorized) { "errors": [{ "title": "Invalid credentials", "status": 401 }] }
```

8. Scrum — Ceremonias y Estimaciones

Ceremonias del Sprint

Ceremonia	Cuándo	Duración	Participantes	Output
Sprint Planning	Inicio del sprint	≤8h/sprint	Todo el equipo	Sprint Goal + Sprint Backlog
Daily Scrum	Cada día	15 min max	Dev team + SM	Sincronización y bloqueos
Sprint Review	Fin del sprint	≤4h/sprint	Equipo + Stakeholders	Demo del incremento
Retrospectiva	Fin del sprint	≤3h/sprint	Todo el equipo	Plan de mejora del proceso
Refinement	Durante el sprint	Continuo	PO + Dev team	Backlog refinado y estimado

Estimaciones — Planning Poker

Se usan números de Fibonacci para reflejar la incertidumbre: la diferencia entre 21 y 34 es mayor que entre 2 y 3. Si hay gran dispersión, se discuten las diferencias hasta llegar a consenso.

Fibonacci: 1 – 2 – 3 – 5 – 8 – 13 – 21 – 34 – ? ↑ ↑ ↑ tareas muy tareas épicas (dividir) chicas medias

Historial de Usuario — Criterios de Aceptación (Gherkin)

```
# Formato Dado/Cuando/Entonces
Escenario 1: Búsqueda con resultados
Dado que estoy en la página de búsqueda
Cuando escribo "zapatillas" en el buscador
Entonces veo hasta 10 productos que contienen "zapatillas"
Y puedo navegar a la siguiente página de resultados
Escenario 2: Búsqueda sin resultados
Dado que estoy en la página de búsqueda
Cuando escribo "xyz123abc" en el buscador
Entonces veo el mensaje "No encontramos resultados"
```

9. Checklist Examen — Antes de Entregar

■ SOLID

- Identifiqué el principio violado y lo nombré correctamente
- Propuse una solución concreta (no solo describí el problema)
- Mencione la señal de alarma específica del principio

■ UML Clases

- Puse multiplicidades en todas las relaciones (1, *, 0..1)
- Diferencié composición (◆) de agregación (<=)
- Separé atributos de métodos en cada clase

■ UML Secuencia

- Los retornos son flechas punteadas, las llamadas son sólidas
- Usé fragmento alt para los casos de error si los hay
- Nombré correctamente los participantes (lifelines)

■ UML Actividad

- Usé Fork/Join si hay actividades en paralelo
- Usé Decisión (rombo) para condiciones sí/sino
- Agregué swim lanes si hay múltiples actores

■ Historias de Usuario

- El actor es un rol de negocio, no "el sistema"
- No presupuse detalles técnicos en la HU
- Los criterios de aceptación usan Dado/Cuando/Entonces

■ Arquitectura

- Justifiqué la elección con contexto (equipo, escala, dominio)
- Mencione los trade-offs de la opción elegida
- En Clean Arch: dependencias apuntan hacia adentro

¡Éxitos en el parcial, Tomás! ■ Recordá: leer el enunciado despacio, identificar qué tipo de respuesta esperan, y siempre justificar tus elecciones.